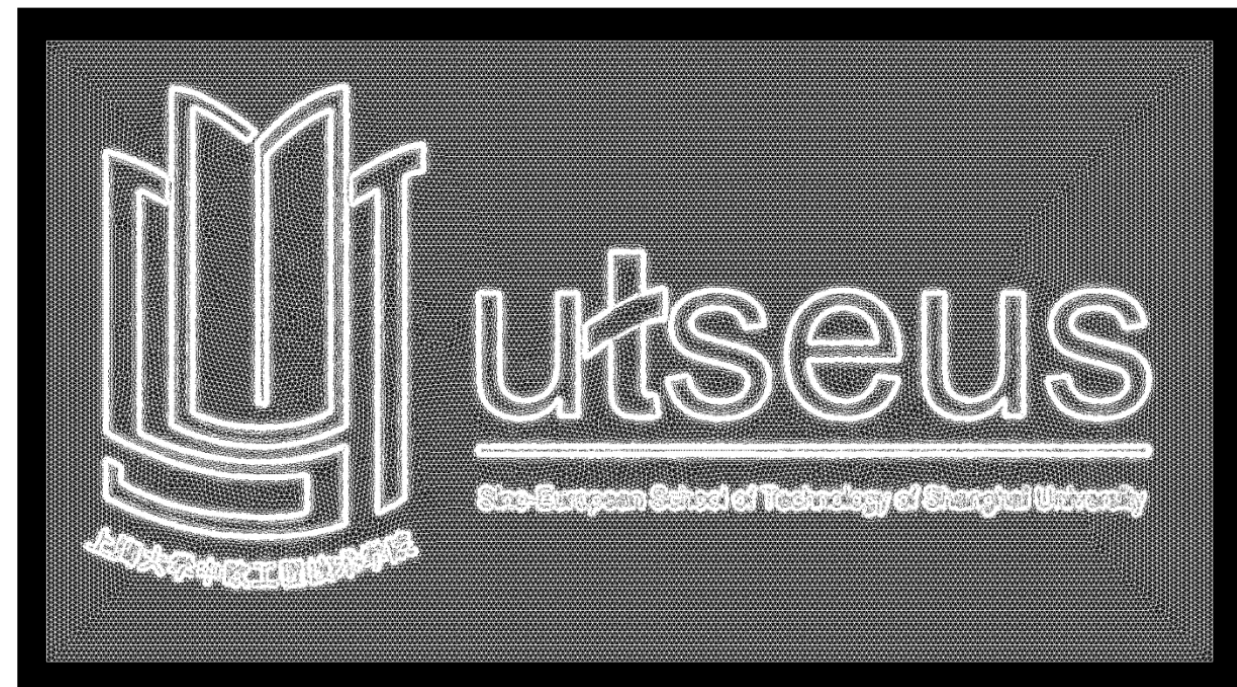




FQ01

MANAGEMENT DE QUALITÉ

Pr. Abel CHEROUAT (abel.cherouat@utt.fr)





La non-qualité dans le management de la qualité

Approche par apprentissage dans le management de la qualité

Intelligence Artificielle dans le Management de la Qualité





Intelligence Artificielle dans le Management de la Qualité

Et si la qualité n'était plus seulement contrôlée... mais prédite ?

Simulation -> Capteurs → Données → IA → Prédiction → Action





Contexte

1. Transformation digitale des entreprises
2. Explosion des données
3. Besoin d'automatisation et de précision
4. **Exigence** accrue en qualité et traçabilité
5. **Limites** des approches traditionnelles
6. Évolution du management de la qualité vers des **systèmes intelligents**

Le **management de la qualité** vise à garantir que les **produits et processus** répondent aux **exigences**.
Avec l'évolution des systèmes industriels (**Industrie 4.0**), l'Intelligence Artificielle (**IA**) on permet améliorer :

1. Détection des défauts
2. Prédiction des anomalies
3. Optimisation des processus
4. Prise de décision basée sur les données





On passe d'une :

- Qualité **Réactive** à une Qualité **Prédictive**.
- Approche traditionnelle basée sur le contrôle à une Approche prédictive et intelligente, améliorant la performance globale des systèmes industriels.

Intelligence Artificielle : ensemble de Techniques permettant à une machine de :

1. Apprendre à partir des données
2. Prendre des décisions
3. Reconnaître des patterns
4. Prédire des non-conformités
5. Détecter les défauts
6. Optimiser les processus qualité

Objectifs de l'IA en qualité :

1. Réduire les défauts
2. Améliorer la prise de décision
3. Automatiser les contrôles
4. Anticiper les problèmes





L'intégration de l'IA permet aujourd'hui d'exploiter de grandes quantités de données issues de capteurs, de systèmes de production et d'historiques de qualité afin d'améliorer la prise de décision.

- Machine Learning
- Deep Learning
- Traitement du langage naturel

Machine Learning : Sous-domaine de l'IA basé sur l'apprentissage à partir de données :

1. Supervisé (classification, régression)
2. Non supervisé (clustering)
3. Apprentissage par renforcement

Les données :

1. Données de production (température, pression, vitesse, humidité...)
2. Données qualité (défauts, non-conformités)
3. Données capteurs (IoT)
4. Historique des inspections



IA versus Gestion de la qualité

Gestion de la qualité Apprentissage

1. Assurer la **conformité** aux exigences
2. Améliorer la **performance des processus**
3. Réduire les **défauts et non-conformités**
4. Augmenter la **satisfaction client**
5. Optimiser les coûts (moins de rebuts, moins de reprises)

Gestion de la qualité IA

1. **Détection automatique** de défauts
2. **Prédiction** de non-conformités
3. **Maintenance** prédictive
4. **Classification** des pièces bonnes / défectueuses

IA = outil d'aide à la décision dans la gestion qualité



Domaines d'application

1. Contrôle qualité automatisé (vision industrielle)
2. Maintenance prédictive
3. Analyse des réclamations clients
4. Optimisation des processus



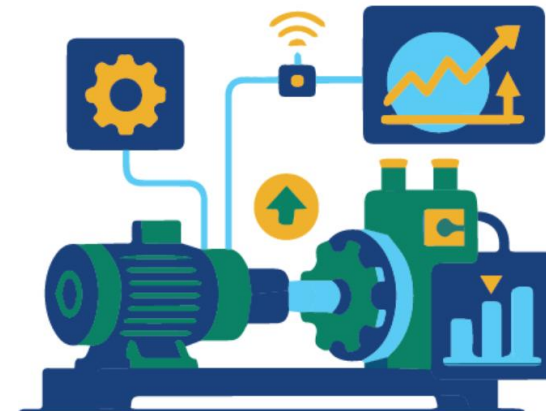
Application 1: Contrôle qualité intelligent (Utilisation du Computer Vision)

1. Caméras + IA détectent les défauts
2. Apprentissage à partir d'images
3. Réduction des erreurs humaines



Application 2 : Maintenance prédictive (utilisation des IoT)

1. Analyse des données machines
2. Détection des anomalies
3. Prévision des pannes





Exemple : Contrôle qualité automatisé dans une usine

Contexte

- Industrie : fabrication de pièces (automobile, électronique)
- Problème :
 - défauts non détectés
 - erreurs humaines
 - contrôle lent

Classification des défauts

1. Défauts de **fabrication** (pièce non conforme aux exigences)
2. Défauts d'**assemblage** (pièce mal assemblée)
3. Défauts liés à la **matière première** (ex : mauvais type d'acier qui provoque plus ou moins de retours élastiques, mauvaises finitions de surface, etc.)
4. Défauts liés à l'**usure** générale des pièces ou des composants (ex : moule qui s'effrite)



Objectif : Améliorer la détection des défauts

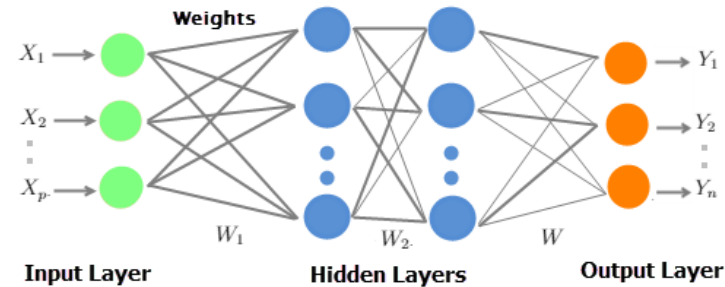
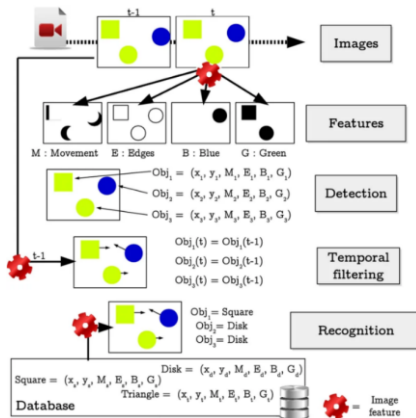


Solution avec IA

1. Caméras haute résolution



2. Algorithme de Computer Vision



3. Modèle de Deep Learning

Réseau de neurones convolutionnel





Inspection: **vision industrielle** + **Deep learning** fonctionnent conjointement :

1- Acquisition d'images

Photos des pièces en temps réel

2- Apprentissage du modèle

Entraînement sur :

- pièces conformes
- pièces défectueuses

Le modèle apprend à reconnaître les défauts

3- Détection automatique

• L'IA analyse chaque pièce

• Classe :

- ✓ conforme
- ✗ non conforme



Avant IA :

- Taux de détection : 85%
- 15% de défauts passent

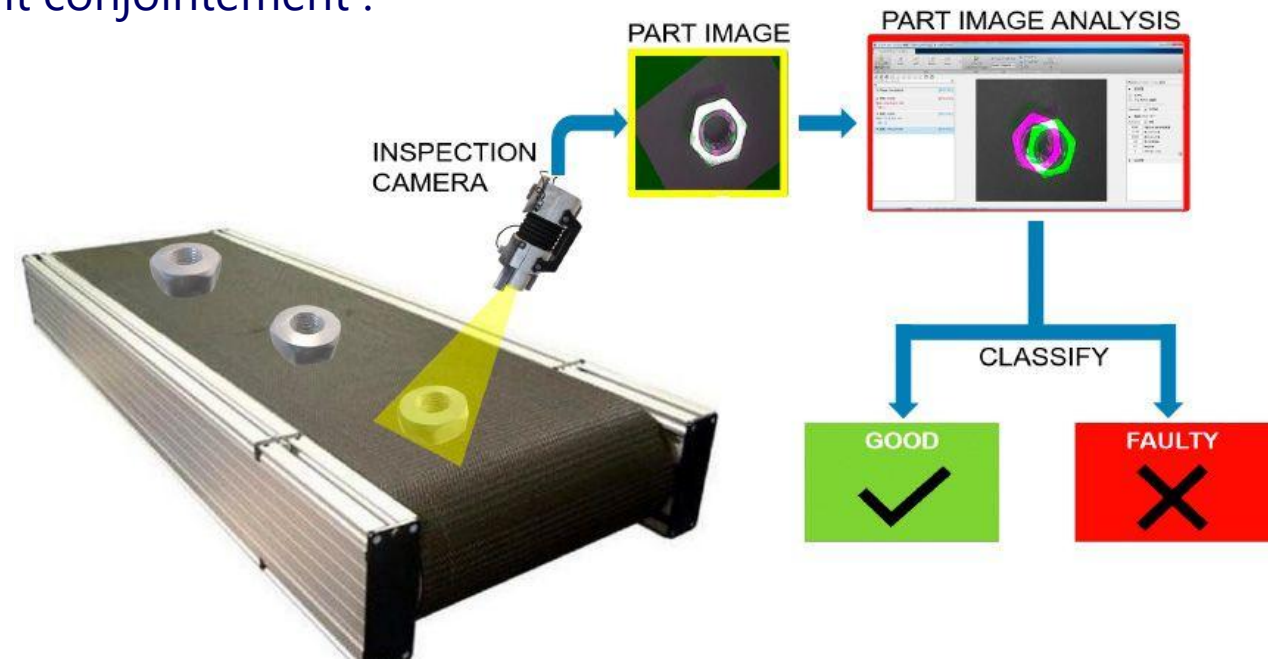
Après IA :

- Taux de détection : 98–99%
- Réduction massive des défauts clients



Impact sur la qualité

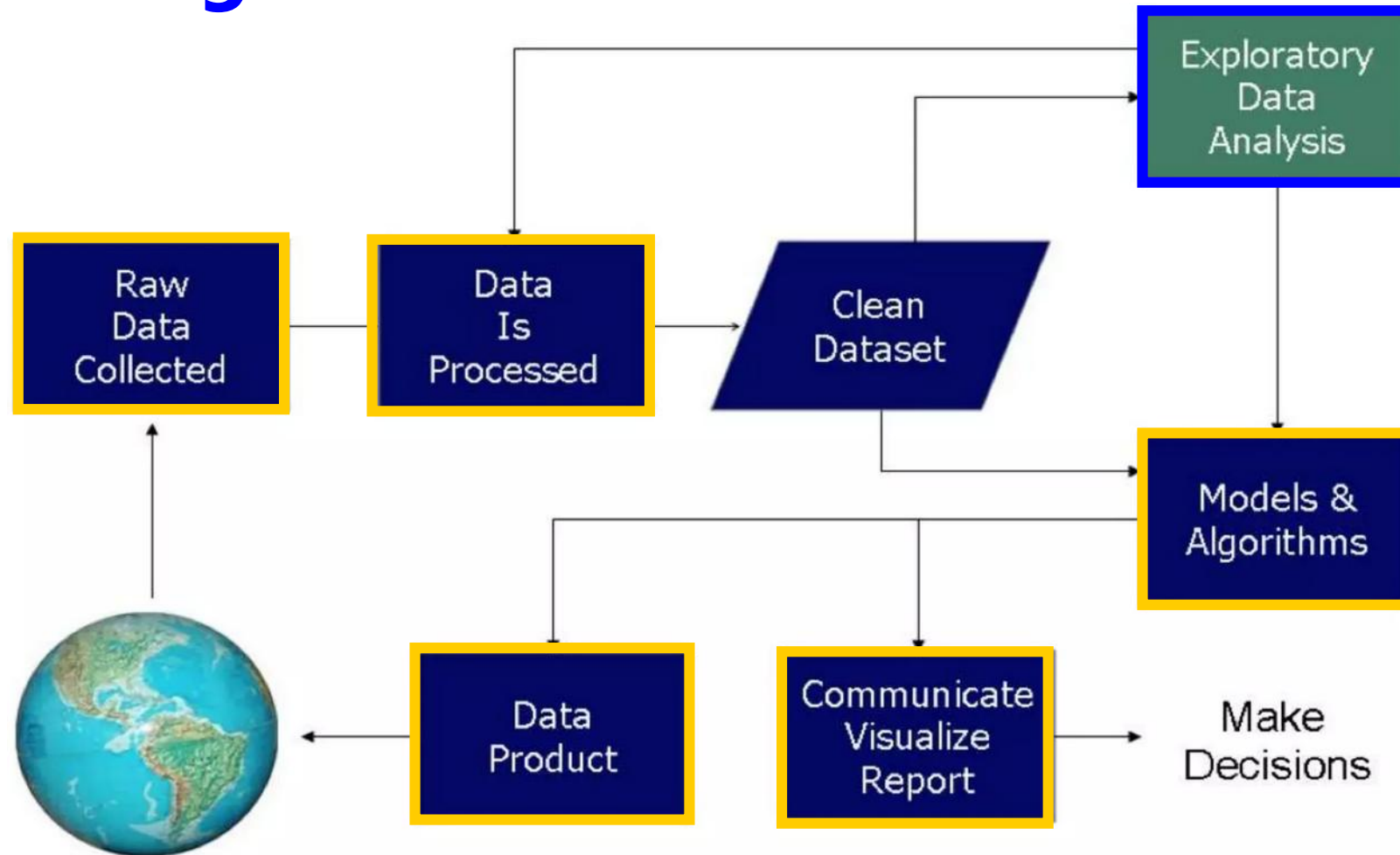
1. ↓ Non-conformités
2. ↓ Retours clients
3. ↓ Coûts de non-qualité
4. ↑ Satisfaction client



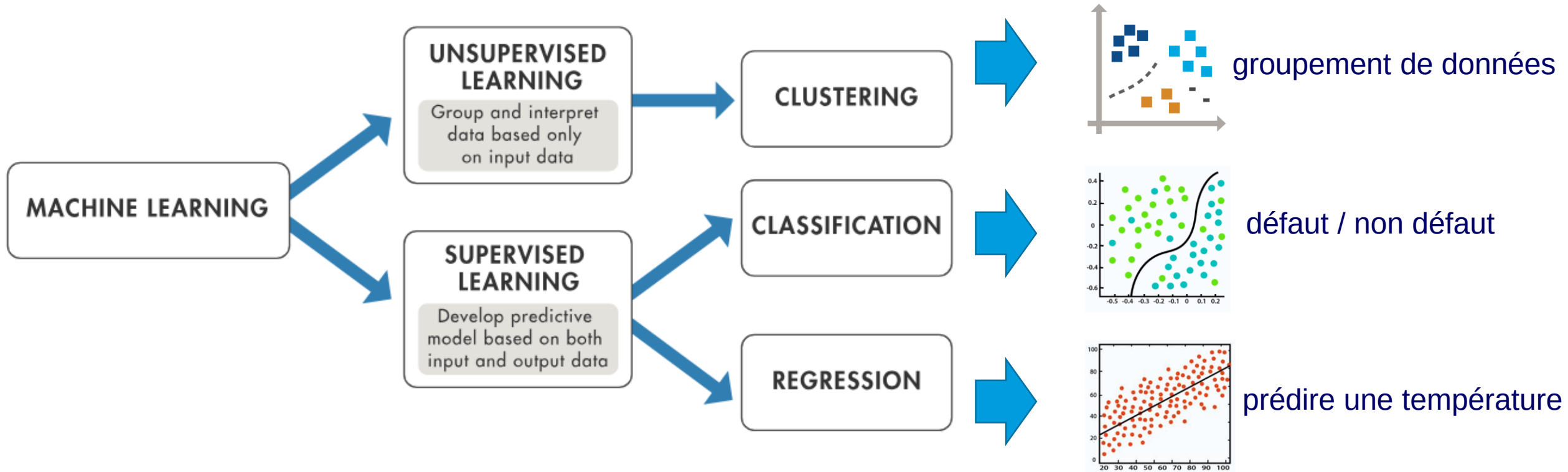


Machine Learning

La **méthodologie du Machine Learning (apprentissage automatique)** correspond à l'ensemble des étapes permettant de **construire, entraîner, valider et déployer** un modèle capable d'apprendre à partir de données.



Machine Learning is a **subset** of Artificial Intelligence that allows systems to **learn from data, identify patterns, and make decisions with minimal human intervention.**



3 types of techniques :

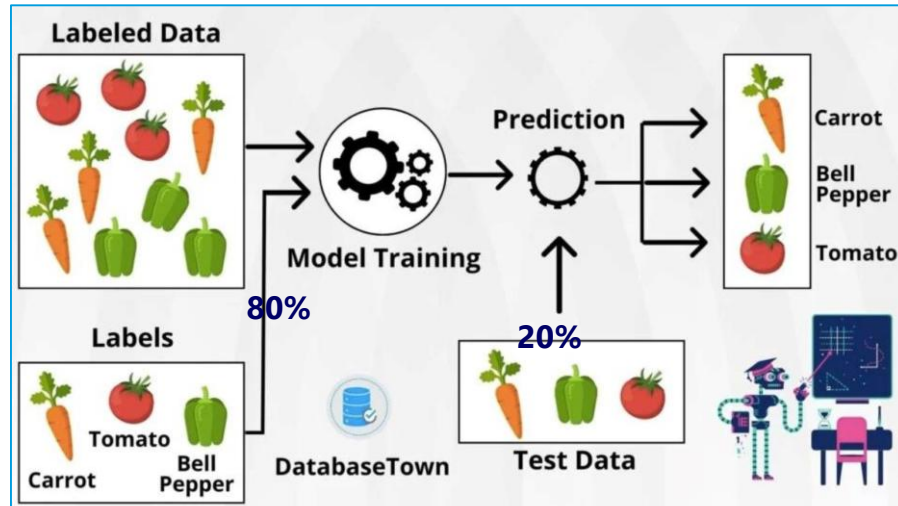
a) **Supervised learning** : train model on known input and output data → predict future outputs

b) **Unsupervised learning** : find hidden patterns or intrinsic structures in input data

c) **Reinforcement learning**: take actions and get rewards

- 监督学习：在已知输入和输出数据上训练模型 → 预测未来的输出
- 无监督学习：在输入数据中发现隐藏的模式或内在结构
- 强化学习：采取行动并获得奖励

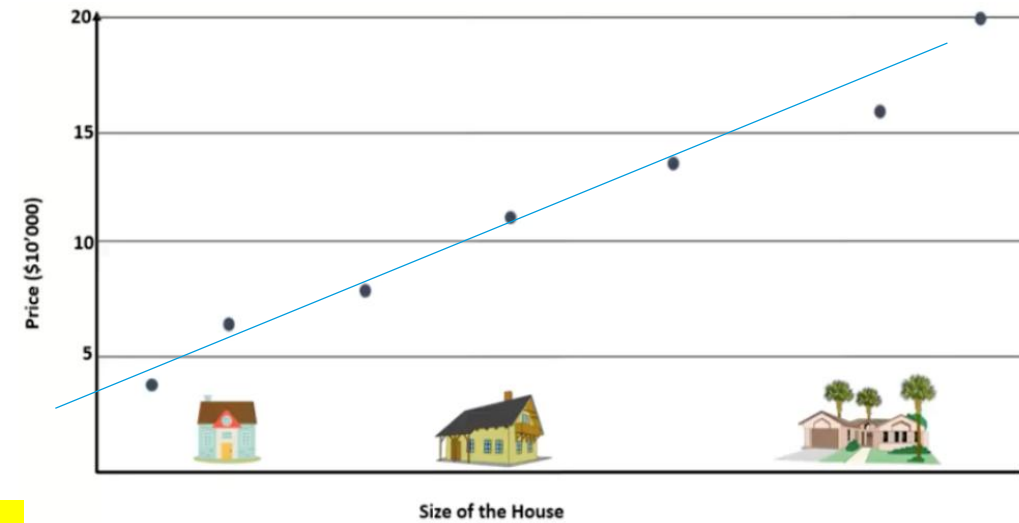
Supervised learning - Classification



Used when the expected result (output variable) is a class (group, category).

Utilisé lorsque le résultat attendu (variable de sortie) est une classe (groupe, catégorie).

Supervised learning - Regression



Used when the expected outcome is a continuous value

Utilisé lorsque le résultat attendu est une valeur continue

Unsupervised learning - Clustering

Input Raw Data



Unlabeled Data

Interpretation

Algorithm

Processing

Dog

Cat

Groups similar data into distinct sets (clusters), based on similarities or distances between data points, thus identifying intrinsic structures and groups within unlabeled data

Regroupe les données similaires en ensembles clusters, en fonction des similarités ou des distances entre les points de données

Reinforcement Learning environment

agent



actions

rewards

observations



Interacts with an environment to learn to make decisions to maximize long-term cumulative rewards

Interagit avec son environnement pour apprendre à prendre des décisions visant à maximiser les récompenses cumulatives à long terme.

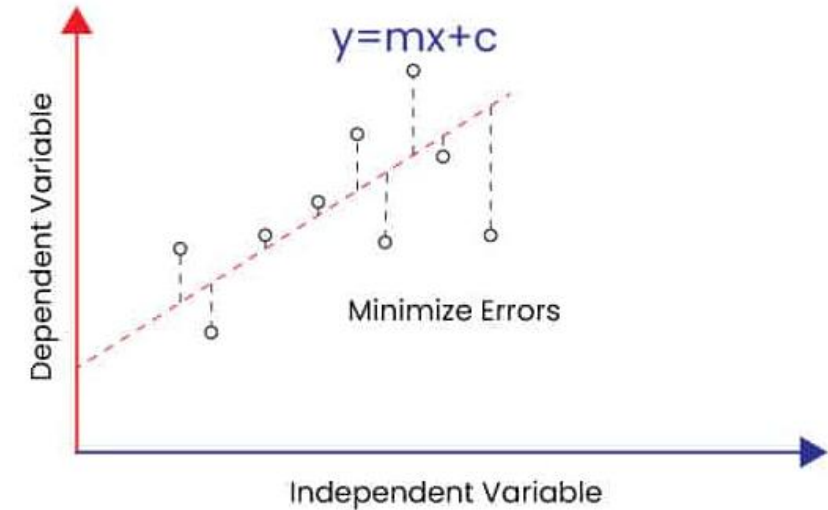


$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_n x_n + \epsilon$$

Linear Regression

Models a **linear relationship** between input features and target.

Best for: Simple, interpretable problems.

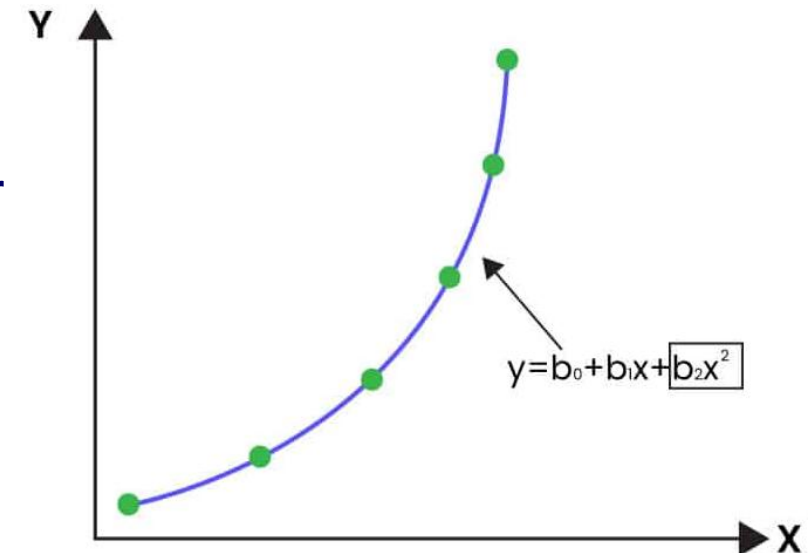


Polynomial Regression

- Transforms original features to include **powers** (x^2, x^3) for modeling **non-linear relationships**.
- Still uses linear regression on transformed features

Transforme les variables initiales pour inclure les puissances (x^2, x^3) afin de modéliser les relations non linéaires.

Utilise une régression linéaire sur les variables transformées.



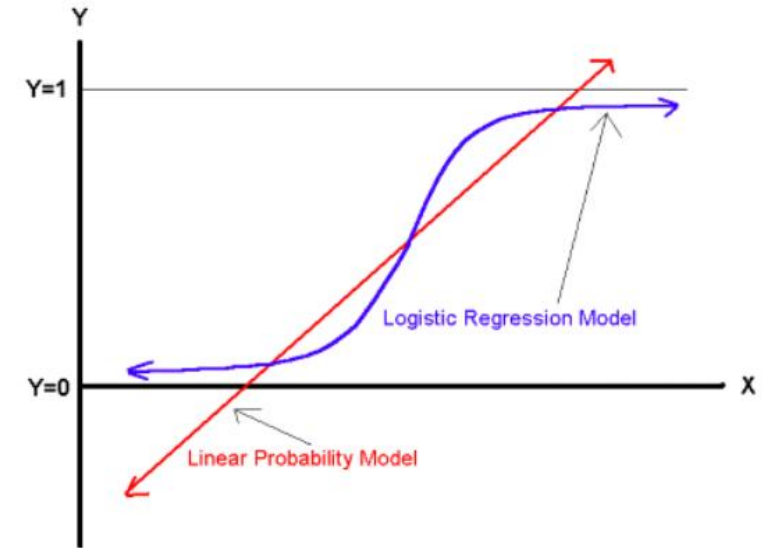


Logistic regression

- Used for **binary classification**, not regression.
- Predicts probabilities using the **sigmoid** function

Méthode d'apprentissage supervisé utilisée pour résoudre des **problèmes de classification** (et non de régression continue) défaut / pas défaut, oui / non, 0 / 1.

Utilisé pour la classification binaire, et non pour la régression.
Prédit les probabilités à l'aide de la fonction sigmoïde.

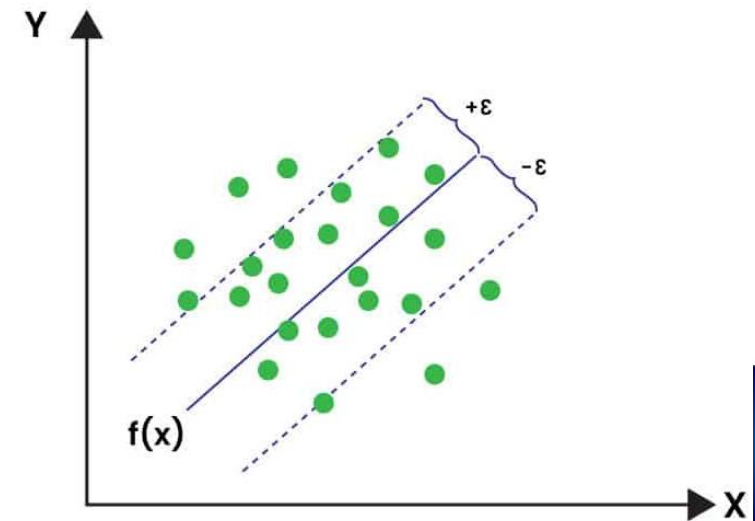


Support Vector Regression (SVR)

- Fit a function **within a margin of tolerance** (epsilon-insensitive).
- Good for **nonlinear** regression with **kernel tricks**. SVR ignores small mistakes, it focuses on big mistakes

Contrairement à une régression classique qui cherche à passer **au plus près de tous les points**, la SVR cherche Une **zone de tolérance** (appelée *marge*) autour de la courbe

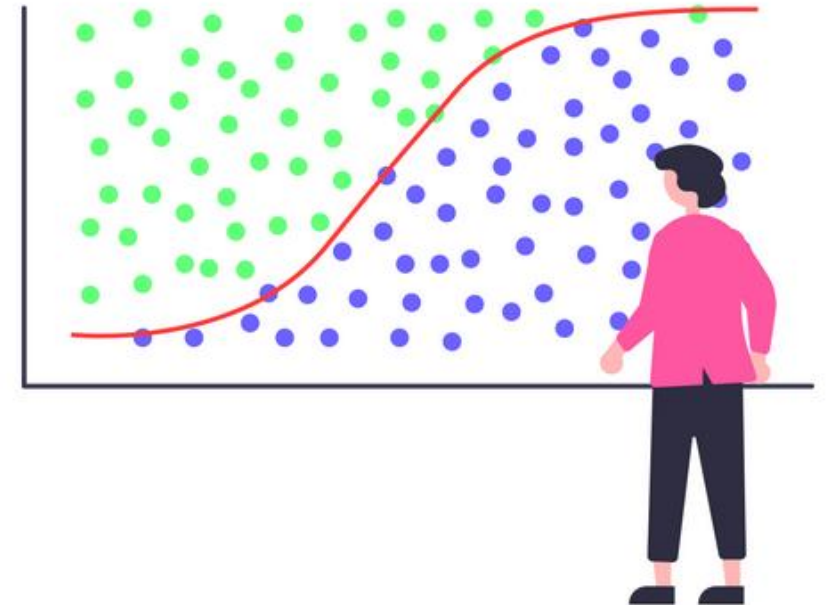
SVR ignore les petites erreurs, il se concentre sur les grosses erreurs





Linear Regression VERSUS Logistic Regression

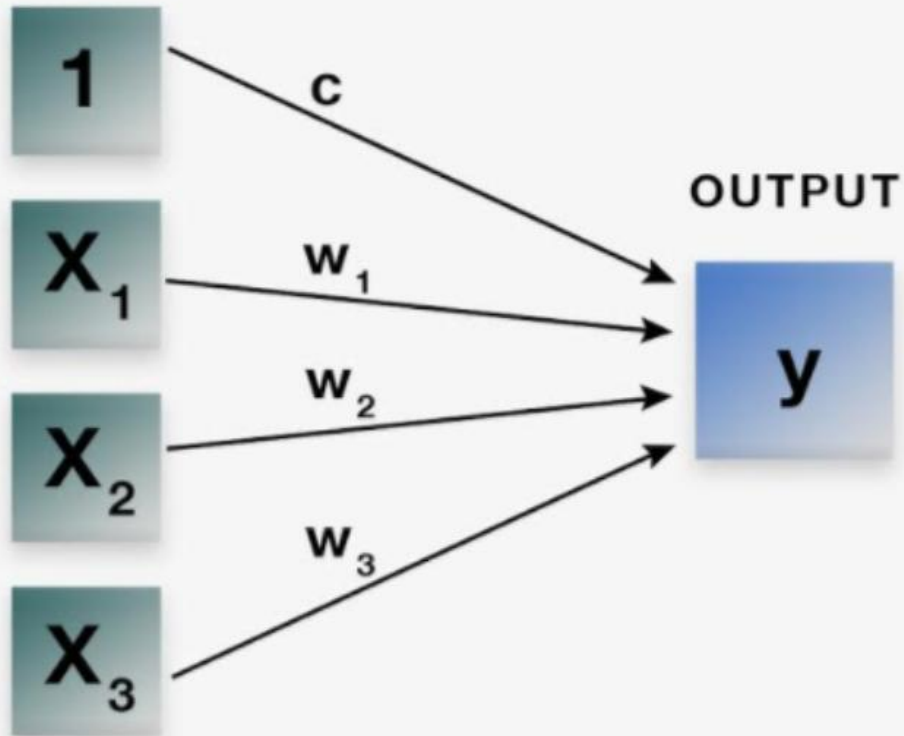
1. Spam detection in emails (spam or not spam)
2. Fraud detection in financial transactions (fraudulent or legitimate)
3. Disease diagnosis based on symptoms (present or absent)



1. Détection des spams dans les courriels (spam ou non)
2. Détection des fraudes dans les transactions financières (frauduleuses ou légitimes)
3. Diagnostic des maladies basé sur les symptômes (présents ou absents)

LINEAR REGRESSION

INPUT FEATURES



We predict a real-valued output y based on a weighted sum of input variables X .

$$y = c + X_1 * w_1 + X_2 * w_2 + X_3 * w_3 + \dots + X_n * w_n$$

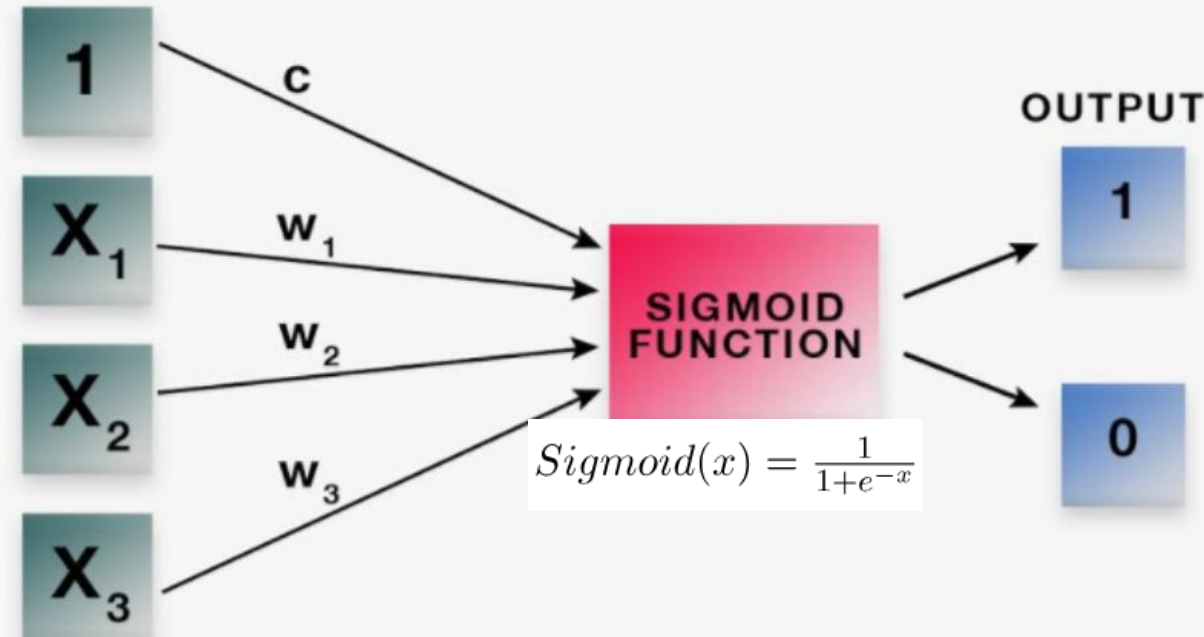


$$y = \text{logistic}(y) = 1 / (1 + e^{-(y = c + X_1 * w_1 + X_2 * w_2 + X_3 * w_3 + \dots + X_n * w_n)})$$

If output $> 0.5 \rightarrow$ classify as 1 (or positive)
if output $< 0.5 \rightarrow$ classify as 0 (or negative)

LOGISTIC REGRESSION

INPUT FEATURES



La **distribution de Bernoulli** est une loi de probabilité très simple qui modélise un **expérience aléatoire à deux issues possibles** : succès ou échec.

Bernoulli distributions characterized by a single parameter :

p : the probability of **success**

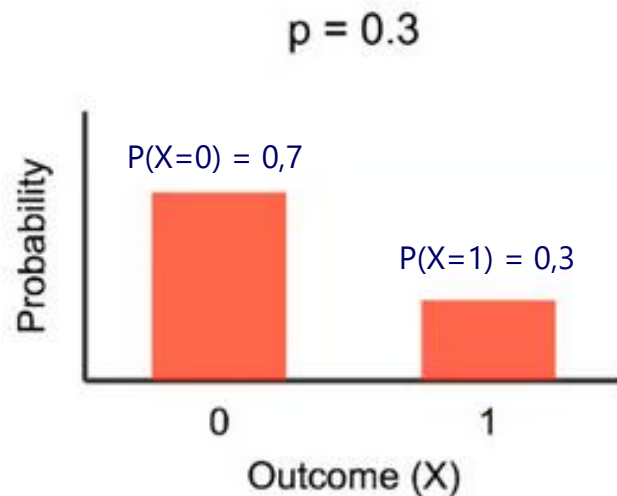
$1 - p$: the probability of **failure**

X is the random variable

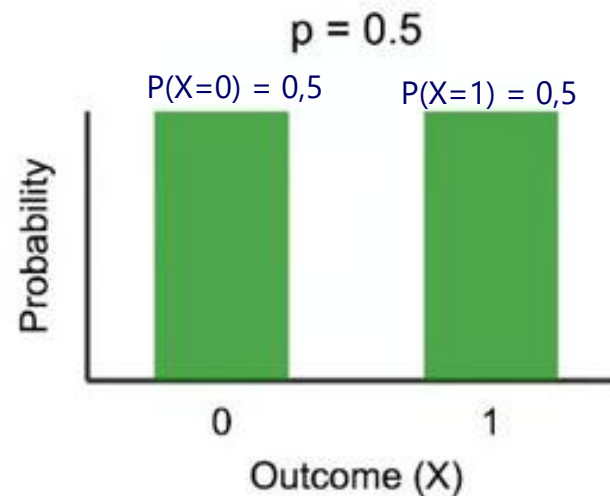
$$P(X = 1) = p, \quad P(X = 0) = 1 - p$$

probabilité de succès

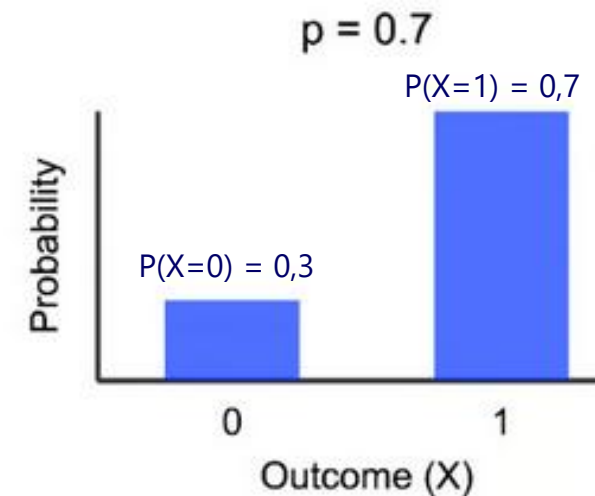
probabilité d'échec



30% chance of success



50% chance of success



70% chance of success

Two bars: one for the probability of failure ($X = 0$) and one for the probability of success ($X = 1$).
The height of these bars changes based on the value of p , sum = 1.



Mean and variance

Mean (Espérance : moyenne)

Mean = p → were to repeat a Bernoulli trial many times, the average outcome converge to p $E[X]=p$

Variance (dispersion des valeurs autour de l'espérance) $\text{var}(X) = \sigma^2 = p(1 - p)$

The variance reaches its maximum when $p = 0.5$ and decreases as p approaches 0 or 1.

Plus la variance est grande → plus l'incertitude est élevée

Bernoulli probability density function :

$$f(y; p) = \begin{cases} 1 - p & \text{for } y = 0 \\ p & \text{for } y = 1 \end{cases}$$

This can also be expressed as $= p^y (1 - p)^{1-y}, \quad y \in \{0, 1\}$

Dans un contrôle qualité :

- $X = 1$ → pièce conforme
- $X = 0$ → pièce défectueuse

Si $p = 0.9$ → 90% des pièces sont conformes → 10% sont défectueuses



L'objectif est de trouver la droite qui représente au mieux les données.

Régression linéaire simple

y : **variable à prédire** (variable dépendante)

x : **variable d'entrée** (variable indépendante)

a ou w: coefficient (pente ou poids)

b : constante (interception)

$$y = ax + b$$

$$y = b_0 + b_1x_1 + b_2x_2 + \dots + b_n x_n$$

Régression linéaire multiple

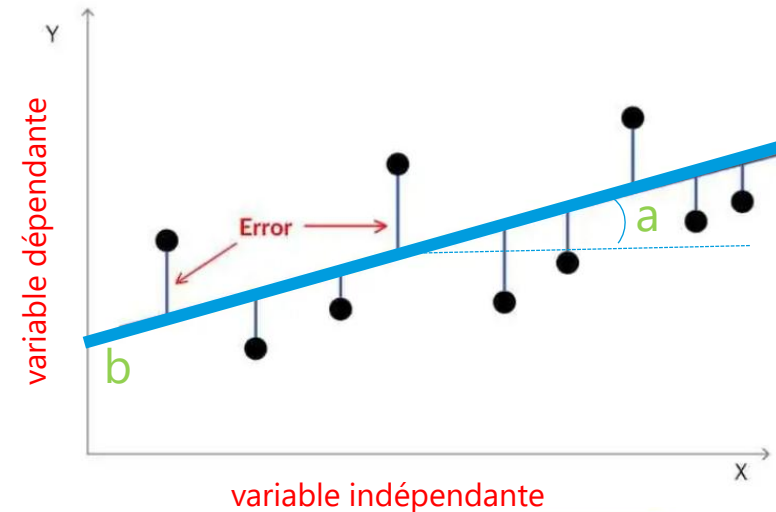
Plusieurs variables :

Objectif du modèle Minimiser l'erreur entre :

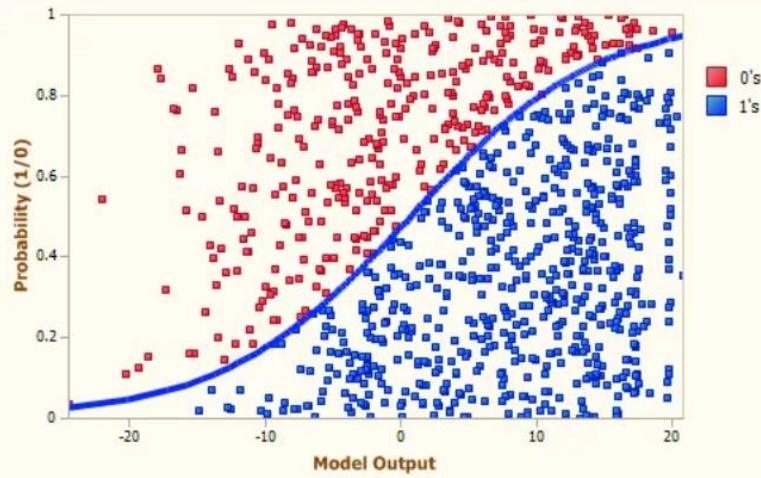
- les valeurs réelles
- les valeurs prédites

Fonction de coût (MSE) Error :

$$MSE = \frac{1}{n} \sum (y_{réel} - y_{prédit})^2$$



To calculate the **intercept b** and **weights w** → **optimization algorithm (gradient descent, Newton-Raphson or libraries like scikit-learn)** that minimizes the **log-loss** (cross-entropy loss) between the model's predicted probabilities and the true labels.



La logistic regression prédit une **probabilité** comprise entre 0 et 1 : fonction sigmoïde (logistique)

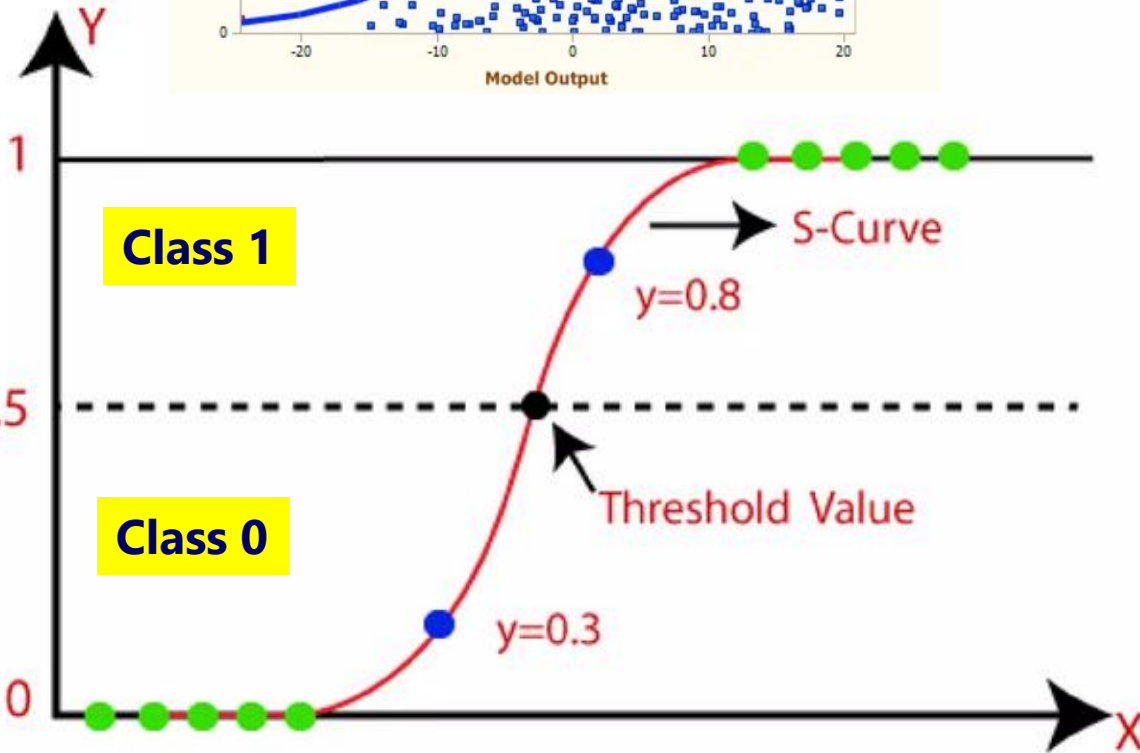
$$(y = 1 | x) = \sigma(z) = \frac{1}{1 + e^{-z}}, \quad \text{where } z = \beta_0 + \beta_1 x_1 + \dots + \beta_n x_n$$

Transforme une combinaison linéaire en probabilité

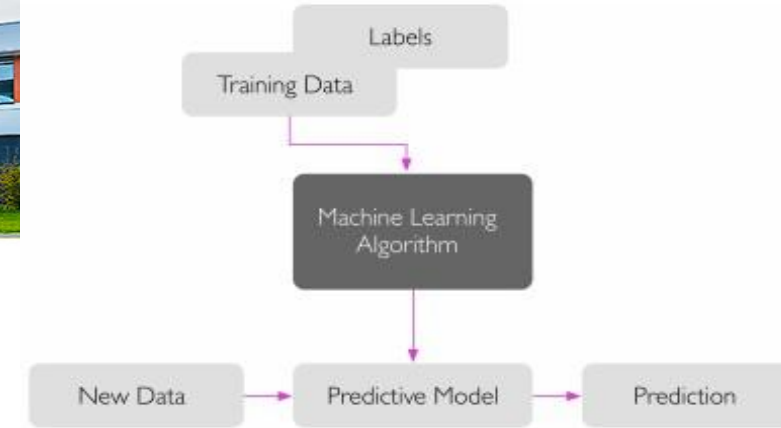
- Si la probabilité $\geq 0.5 \rightarrow$ classe 1 (or positive)
- Si la probabilité $< 0.5 \rightarrow$ classe 0 (or negative)

Le seuil peut être ajusté selon le problème.

Exemple : $P(\text{défaut} = 1 | \text{température, humidité}) = 0.8$
 $\rightarrow$ 80% de chances d'avoir un défaut.



- The curve **smoothly transitions** from 0 to 1.
- At $x = 0 \rightarrow \sigma(x)=0.5$.
- The function is **symmetric around the origin**.



Le Machine Learning suit un processus logique :

1. Collecte des données

2. Préparation des données

Nettoyer et organiser les données

5. Entraînement du modèle

Apprendre à partir des données

6. Évaluation du modèle

Mesurer la performance

Données → Modèle → Prédiction → Amélioration

3. Analyse exploratoire

Comprendre les relations dans les données

4. Choix du modèle

Choisir l'algorithme adapté

7. Amélioration (itération)

Optimiser le modèle

8. Déploiement

Utiliser le modèle en situation réelle

Exemple (qualité)

1. Taux de **défauts**

2. Formation

3. **Incidents** machines

→ Plus les données sont **fiables**, meilleur sera le modèle



1 - Problem Understanding

- **Define the Problem:** What is the target variable (dependent variable)? What are the predictors (independent variables)?
- **Understand the Domain:** Is there any domain knowledge about the relationships between variables?

2 - Understanding the Data

- Is the relationship linear or non-linear?
- Are there any outliers or noise in the data?
- Are there multiple features or just one?
- Is the target variable continuous (suitable for regression) or categorical?



3 - Choosing the Right Regression Algorithm

- **Linear Regression:** If there is a linear relationship between the independent variables and the target variable.
- **Polynomial Regression:** Used when the relationship is non-linear, but can be modeled using a polynomial expression.
- **Support Vector Regression (SVR):** SVR works well when you have a non-linear relationship, and it can handle high-dimensional feature spaces.
- **Neural Networks:** When you have a large dataset and complex relationships, deep learning models (e.g., neural networks) can be used for regression tasks.



4 - Splitting the Data

- **Training Set:** Used to train the model.
- **Test Set:** Used to evaluate the model's performance.

A common practice is to use an **80-20 split** (80% of data for training, 20% for testing), but the split ratio can vary depending on the data size.

5 - Training the Model

- Feeding the data to the model
- Adjusting the model parameters to minimize the error (e.g., using methods like gradient descent)
- Finding the best-fitting line or curve (in case of non-linear models)

6 - Model Evaluation

• Evaluate on Test Data:

- Assess the performance using appropriate metrics:
 - **MAE** (Mean Absolute Error)
 - **MSE** (Mean Squared Error)
 - **RMSE** (Root Mean Squared Error)
 - **R^2 (Coefficient of Determination)**
 - **Adjusted R^2**
- Compare the **predicted vs actual** values using a scatter plot or residual plot.





A **loss function** in machine learning is a mathematical function that measures the **difference between the model's prediction and the actual target value**. The goal of training a model is to minimize the loss function, which indicates better performance.

Regression Loss Functions

1. Mean Squared Error (MSE):

1. Commonly used for regression tasks.
2. Measures the average of the squared differences between predicted and actual values.

Penalizes larger errors more.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

Diagram illustrating the Mean Squared Error (MSE) formula. The formula is enclosed in a blue box. Arrows point from labels to parts of the formula: "true label" points to y_i , "predicted value" points to $\hat{y}_i$, and "number of data points" points to n .

2. Mean Absolute Error (MAE):

- Also used in regression tasks.
- Measures the average of the absolute differences between predicted and actual values.

More robust to outliers than MSE, but it may be less sensitive to large errors.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$



Classification Loss Functions

La **Cross-Entropy Loss** sert à mesurer : **à quel point un modèle se trompe** dans un problème de **classification**

$$L = - (y \log(p) + (1 - y) \log(1 - p))$$

y : vraie valeur (0 ou 1)

p : probabilité prédite

Le modèle donne une **probabilité** :

- Prédit : 0.9 (90% non conforme)
 - Réalité : 1 (non conforme)
- Bonne prédiction → erreur faible

• Prédit : 0.1

• Réalité : 1

→ Mauvaise prédiction → erreur forte



1. Cross-Entropy Loss (Log Loss fonction de perte):

- Commonly used for classification tasks, especially binary or multi-class classification.
- Measures the difference between two probability distributions: the true distribution (true labels) and the predicted distribution (model's output probabilities).
- Formula for binary classification:
 - For binary classification (e.g., spam vs not spam).
 - Penalizes confident wrong predictions heavily.

$$\text{Cross-Entropy} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

true label (0 or 1)

Predicted score

```
import numpy as np

def binary_cross_entropy(y_true, y_pred, epsilon=1e-15):
    # Avoid log(0) by clipping predictions
    y_pred = np.clip(y_pred, epsilon, 1 - epsilon)
    loss = - (y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))
    return np.mean(loss)
```

`np.clip(a, a_min, a_max)` function is used to Clip (limit) the values in an array.
Given an interval, values outside the interval are clipped to the interval edges.

`np.clip(in_array, a_min = 2, a_max = 6)`

Input array : [1, 2, 3, 4, 5, 6, 7, 8] → Output array : [2 2 3 4 5 6 6 6]

2. Hinge Loss:

- Used for **support vector machines** (SVM) in classification tasks.
- The hinge loss function penalizes misclassifications, with a margin between the correct and predicted class.
- Utilisée pour : les **problèmes de classification binaire**
- L'objectif n'est pas seulement de bien classer, mais de **classer avec une marge de sécurité**

y = vraie classe (+1 ou -1) $f(x)$ = prédiction du modèle (score)

Cas 1 : Bonne prédiction avec marge

$y \cdot f(x) \geq 1$ **Perte = 0** Le modèle est satisfait

Cas 2 : Bonne mais faible confiance

$0 < y \cdot f(x) < 1$ **Perte > 0** Le modèle veut améliorer la marge

Cas 3 : Mauvaise prédiction

$y \cdot f(x) < 0$ **Perte élevée** Grosse pénalité

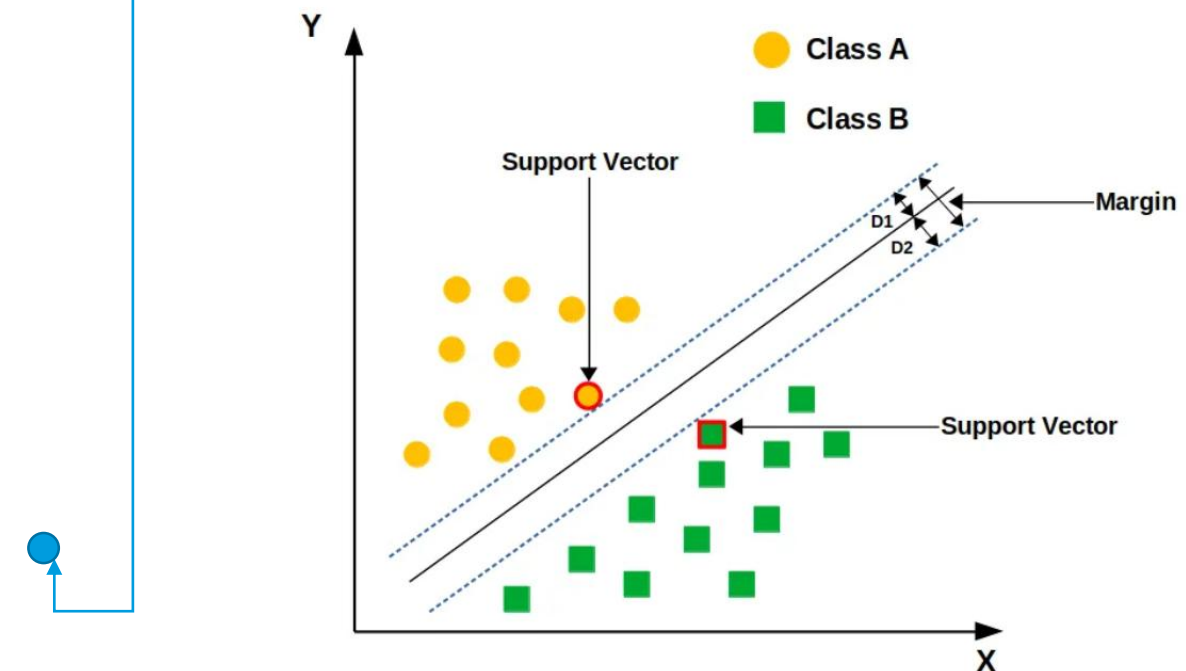
```
import numpy as np

def hinge_loss(y_true, y_pred):
    # y_true must be -1 or +1
    loss = np.maximum(0, 1 - y_true * y_pred)
    return np.mean(loss)
```

$$L(y, \hat{y}) = \max(0, 1 - y \cdot \hat{y})$$

Predicted score

true label (-1 or 1)





3. Huber Loss:

- A combination of **MSE** and **MAE**, it is less sensitive to outliers than MSE and more robust than MAE.
- It behaves like **MSE** when the error is small (to ensure smooth gradients).
- It behaves like **MAE** when the error is large (to reduce the effect of outliers)

$$L_{\delta}(y, \hat{y}) = \begin{cases} \frac{1}{2} (y - \hat{y})^2 & \text{if } |y - \hat{y}| \leq \delta \\ \delta (|y - \hat{y}| - \frac{1}{2}\delta) & \text{otherwise} \end{cases}$$

δ is a threshold parameter that controls the transition between the squared error and the linear error.

◆ Choosing δ

- A good default is 1.0, but it depends on the scale of your data.
- Small δ : more like MAE
- Large δ : more like MSE

```
import numpy as np

def huber_loss(y_true, y_pred, delta=1.0):
    error = y_true - y_pred
    is_small_error = np.abs(error) <= delta
    squared_loss = 0.5 * error**2
    linear_loss = delta * (np.abs(error) - 0.5 * delta)
    return np.where(is_small_error, squared_loss, linear_loss)
```


Evaluation Metrics for Regression Models



- **Mean Absolute Error (MAE):** The average absolute difference between the predicted values and actual values.

- Robust to outliers, but may not penalize large errors heavily.
- Easy to interpret (same units as the output).

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **Mean Squared Error (MSE):** The average of the squared differences between the predicted values and actual values. It penalizes large errors more heavily than MAE.

- Sensitive to outliers.
- Useful when large errors are particularly bad.

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

- **Root Mean Squared Error (RMSE):** The square root of MSE. It gives the error in the same units as the target variable, making it easier to interpret.

Easier to interpret than MSE.

$$RMSE = \sqrt{MSE}$$

- **R-squared (R^2):** A measure of how well the model fits the data. It ranges from 0 to 1, where 1 indicates a perfect fit and 0 indicates that the model explains none of the variance in the data.

- 1: perfect prediction
- 0: model predicts as poorly as mean
- < 0 : model is worse than just predicting the mean

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$



Evaluation Metrics for Regression Models via python

```
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
```

```
y_true = [3.0, -0.5, 2.0, 7.0]
```

```
y_pred = [2.5, 0.0, 2.1, 7.8]
```

```
mae = mean_absolute_error(y_true, y_pred)
```

```
mse = mean_squared_error(y_true, y_pred)
```

```
rmse = mean_squared_error(y_true, y_pred, squared=False)
```

```
r2 = r2_score(y_true, y_pred)
```

```
print(f"MAE: {mae:.2f}")
```

```
print(f"MSE: {mse:.2f}")
```

```
print(f"RMSE: {rmse:.2f}")
```

```
print(f"R²: {r2:.2f}")
```

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

$$RMSE = \sqrt{MSE}$$

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

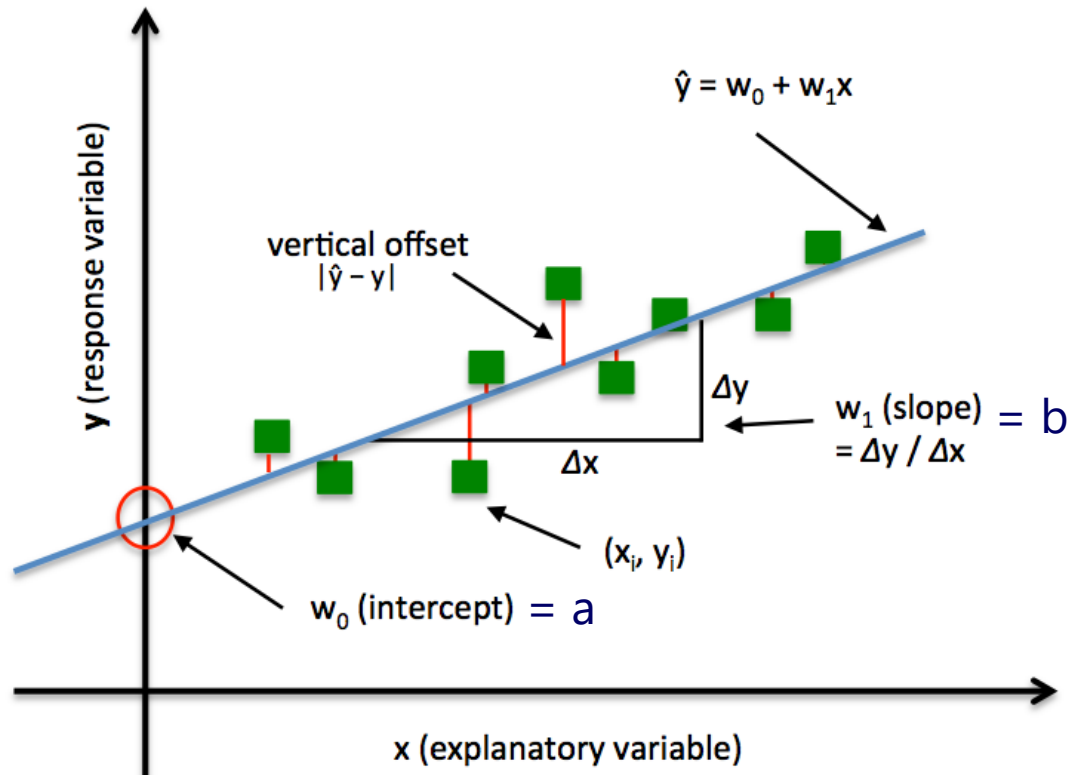


Linear Regression

Estimate **slope (b)** and **intercept (a)** of the linear equation predicting an **outcome variable Y**, based on values of a **predictor variable X** by the **least squares method** (minimisation of the sum of squares of deviations from a straight line).

The intercept a , is the predicted value of Y when $X=0$.
The slope b , is the average change in Y for every one unit increase in X .

$$Y = a + bx$$



$$b = \frac{\sum_{i=1}^n (x_i - \bar{x})(Y_i - \bar{Y})}{\sum_{i=1}^n (x_i - \bar{x})^2}$$

$$a = \bar{Y} - b\bar{x}$$

mean of the y-values

mean of the x-values

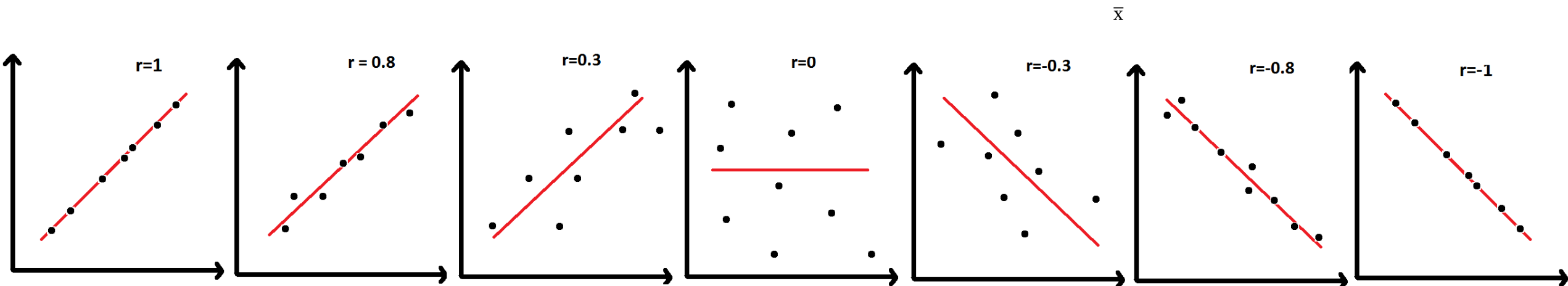


Pearson's Product Moment Correlation Coefficient (denoted **r**), measures the **linear relationship** between two continuous variables.

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(Y_i - \bar{Y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2 \sum_{i=1}^n (Y_i - \bar{Y})^2}}$$

r value	Interpretation
$r = 1$	Perfect positive linear correlation
$1 > r \geq 0.8$	Strong positive linear correlation
$0.8 > r \geq 0.4$	Moderate positive linear correlation
$0.4 > r > 0$	Weak positive linear correlation
$r = 0$	No correlation
$0 > r \geq -0.4$	Weak negative linear correlation
$-0.4 > r \geq -0.8$	Moderate negative linear correlation
$-0.8 > r > -1$	Strong negative linear correlation
$r = -1$	Perfect negative linear correlation

The **correlation coefficient r** takes its values in the interval $[-1; 1]$





```
import numpy as np
from scipy.stats import pearsonr
```

```
# Example data
```

```
x = [10, 20, 30, 40, 50]
```

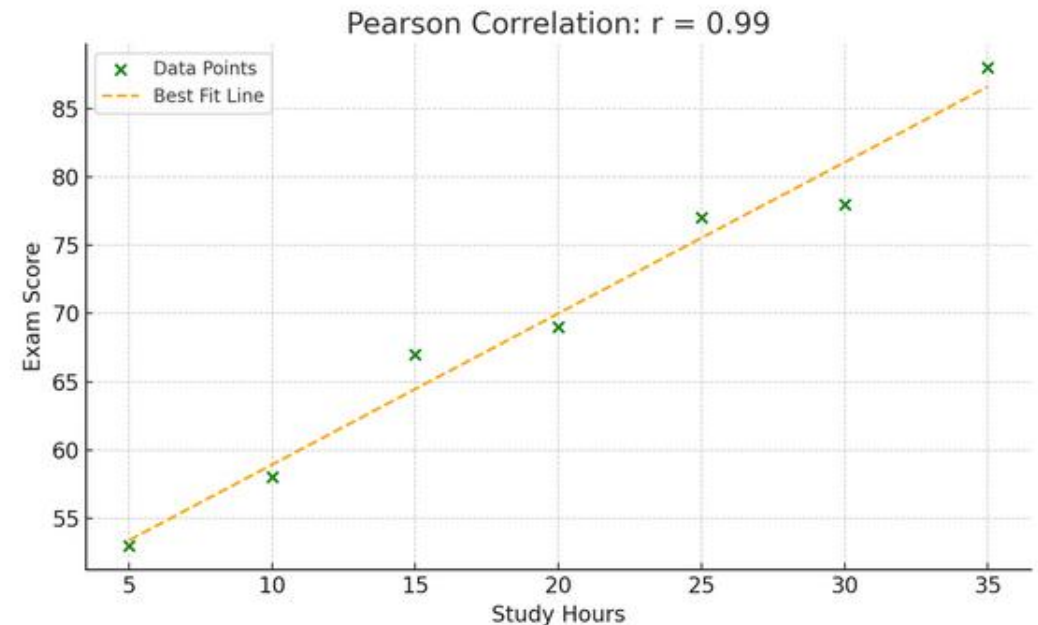
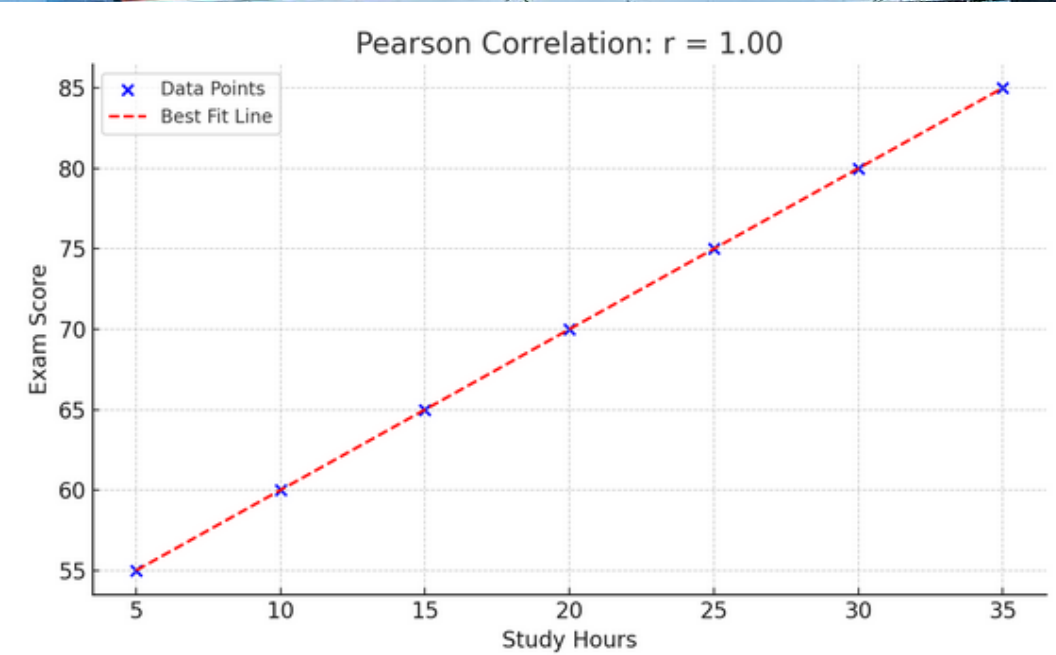
```
y = [15, 25, 35, 45, 55]
```

```
# Pearson correlation
```

```
r, p_value = pearsonr(x, y)
```

```
print(f"Pearson correlation coefficient: {r}")
```

```
print(f"P-value: {p_value}")
```





Exemple 1: Prédire le **taux de défauts (%)** à partir de la formation et les incidents machines.
Modèle de régression linéaire pour comprendre l'impact des variables sur la qualité

Formation (h)	Nombre Incidents	Défauts (%)
2.0	7	5.0
1.5	9	5.6
1.0	12	6.4
0.7	14	7.2
0.5	17	7.9
0.3	21	8.5

1. Charger les données dans Python
2. Construire un modèle de **régression linéaire**
3. Prédire le **taux de défauts** pour :
 Formation = 0.4
 Incidents = 18
4. Interpréter le résultat



1. Charger les données dans Python

```
import pandas as pd

# Création du dataset
data = {
    "formation": [2.0, 1.5, 1.0, 0.7, 0.5, 0.3],
    "incidents": [7, 9, 12, 14, 17, 21],
    "defauts": [5.0, 5.6, 6.4, 7.2, 7.9, 8.5]
}
```

On stocke ce tableau dans une variable appelée **df**



```
df = pd.DataFrame(data)
```

Convertit le dictionnaire en **tableau structuré**





2. Construire un modèle de régression linéaire

```
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
```

```
# Variables explicatives (X) et cible (y)
X = df[["formation", "incidents"]]
y = df["defauts"]
```

```
# Séparation train/test
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
# Création du modèle
```

```
model = LinearRegression()
```

```
# Entraînement
```

```
model.fit(X_train, y_train)
```

```
# Résultats
```

```
print("Intercept :", model.intercept_)
print("Coefficients :", model.coef_)
```

Séparer les données en deux parties :
Données d'entraînement (train)
Données de test (test)

contrôler le hasard dans les
opérations aléatoires
42 : Fixer le hasard pour
reproduire exactement le
même découpage



3. Prédiction : faire une prédiction avec un modèle entraîné

```
nouvelle_donnee = [[0.4, 18]] # faible formation, incidents élevés  
  
prediction = model.predict(nouvelle_donnee)  
  
print("Taux de défauts prédit :", prediction[0])
```

Le modèle utilise la formule apprise :

$$y = -2.1 \times \text{formation} + 0.18 \times \text{incidents} + 9.8$$

Prédire le taux de défauts dans les situations suivantes :

1. Formation = 1.5 ; Incidents = 10
2. Formation = 0.2 ; Incidents = 20
3. Formation = 2.0 ; Incidents = 6

Sous forme de tableau `[[valeur1, valeur2]]`

- formation = 0.4
 - incidents = 18
- calcule le résultat





4. Visualisation simple

les **coefficients (poids)** du modèle de régression

```
print("Coefficient formation :", model.coef_[0])  
print("Coefficient incidents :", model.coef_[1])
```

`[-2.1, 0.18]`

$$y = \underbrace{-2.1}_{w_0} \times \text{formation} + \underbrace{0.18}_{w_1} \times \text{incidents} + \underbrace{9.8}_b$$

- Coefficient formation → négatif
→ Plus de formation = moins de défauts
- Coefficient incidents → positif
→ Plus d'incidents = plus de défauts

```
import matplotlib.pyplot as plt  
  
plt.scatter(df["incidents"], df["defauts"])  
plt.xlabel("Incidents machines")  
plt.ylabel("Taux de défauts")  
plt.title("Relation incidents - défauts")  
plt.show()
```



le modèle permet de :

1. Comprendre les relations entre variables
2. Prédire les défauts futurs
3. Aider à la décision

Grâce à ce modèle, on peut :

- Anticiper les problèmes qualité
- Planifier la formation
- Réduire les incidents





Exemple 2 : Prédire défaut / non défaut (Logistic Regression)

1. Données

On transforme le problème en **classification** :

Si défaut > 7% → 1 (défectueux)

Sinon → 0 (conforme)

```
import pandas as pd

data = {
    "formation": [2.0, 1.5, 1.0, 0.7, 0.5, 0.3],
    "incidents": [7, 9, 12, 14, 17, 21],
    "defauts": [5.0, 5.6, 6.4, 7.2, 7.9, 8.5]
}

df = pd.DataFrame(data)

# Création de la variable cible (classification)
df["classe"] = (df["defauts"] > 7).astype(int)
```

convertir des données en nombres entiers (integers)



2. Modèle Logistic Regression

```
from sklearn.linear_model import LogisticRegression
```

```
# Variables explicatives
```

```
X = df[["formation", "incidents"]]
```

```
# Variable cible (0 ou 1)
```

```
y = df["classe"]
```

```
# Création du modèle
```

```
model = LogisticRegression()
```

```
# Entraînement
```

```
model.fit(X, y)
```





3. Prédiction

```
# Nouvelle situation
```

```
nouvelle_donnee = [[0.4, 18]]
```

```
prediction = model.predict(nouvelle_donnee)
```

```
proba = model.predict_proba(nouvelle_donnee)
```

```
print("Classe prédite :", prediction[0])
```

```
print("Probabilité défaut :", proba[0][1])
```

obtenir les **probabilités de chaque classe**
(conforme / non conforme)

classe (0 ou 1)

probabilité

- 0.08 → probabilité **classe 0 (conforme)**
- 0.92 → probabilité **classe 1 (défaut)** 92% de chance que la pièce soit défectueuse

4. Interprétation

- Classe prédite : **1**
- Probabilité : **0.92**

- Forte probabilité de défaut
- Situation critique



5. Comprendre le modèle

```
print("Coefficient formation :", model.coef_[0][0])  
print("Coefficient incidents :", model.coef_[0][1])
```

Si Formation → coefficient négatif
→ protège contre les défauts
Si Incidents → coefficient positif
→ augmente le risque

6. Visualisation

```
import matplotlib.pyplot as plt  
  
plt.scatter(df["incidents"], df["classe"])  
plt.xlabel("Incidents")  
plt.ylabel("Classe (0=OK, 1=Défaut)")  
plt.title("Classification défaut")  
plt.show()
```



La régression logistique permet de :

1. Prédire un défaut (oui/non)
2. Estimer une probabilité
3. Aider à la décision qualité



Scikit-Learn for Regression

The name "scikit-learn" originated from the combination of "SciPy Toolkit" and the focus on machine learning tasks.

Scikit-Learn, also known as sklearn, is a popular open-source library for machine learning in Python.

Scikit-Learn offers a wide range of machine learning algorithms for regression, classification, clustering, and more. It provides tools for evaluating and comparing model performance, including cross-validation, R^2 metric, mean square error, and more.

Let's look at the dataset below containing information about cars:

EXAMPLE : We can predict a car's CO₂ emissions based on engine volume.

However, using multiple regression, we can incorporate more explanatory variables, such as the car's weight, to improve the accuracy of CO₂ emissions prediction.

Car	Model	Volume	Weight	CO2
Toyota	Aygo	1000	790	99
Mitsubishi	Space Star	1200	1160	95
Skoda	Citigo	1000	929	95
Fiat	500	900	865	90
Mini	Cooper	1500	1140	105
VW	Up!	1000	929	105
Skoda	Fabia	1400	1109	90
Mercedes	A-Class	1500	1365	92
Ford	Fiesta	1500	1112	98


```
1 import pandas as pd
2 import numpy as np
3 from sklearn import linear_model
4
5 df = pd.read_csv('carsemission.csv')
6
7 X = df[['Weight', 'Volume']]
8 y = df['CO2']
9 model = linear_model.LinearRegression()
10 model.fit(X, y)
11
12 print(f"w1 w2: {model.coef_}")
13 print(f"w0: {model.intercept_}")
14 print(f"R2 score en utilisant model.score : {model.score(X,y):.2f}")
15
```

w1 w2: [0.00755095 0.00780526]
w0: 79.69471929115939
R2 score en utilisant model.score : 0.38

We use the `LinearRegression` method to create a linear regression object called `model`.

`model.fit(X, y)` fits the model to the data.

Note: For this two-feature model, we need to determine w_0 , w_1 , w_2 .

`model.coef_` returns the coefficients w_1 , w_2 for the Weight and Volume features, respectively.

`model.intercept_` returns w_0 .



```

16 y_pred = model.predict(X)
17 print(f"min de y: {np.min(y)}, max de y: {np.max(y)}, moyenne de y: {np.mean(y):.2f}")
18 print(f"min de y_pred: {np.min(y_pred):.2f}, \
19 max de y_pred: {np.max(y_pred):.2f}, moyenne de y_pred: {np.mean(y_pred):.2f}")
20
21 from sklearn.metrics import mean_squared_error, r2_score
22 print()
23 print(f"R2 score en utilisant r2_score : {r2_score(y, y_pred):.2f}")
24 print(f"Training RMSE : {np.sqrt(mean_squared_error(y, y_pred)):.2f}")
25

```

`model.predict` expects a 2D array as input (each row is a new sample)

```

min de y: 90, max de y: 120, moyenne de y: 102.03
min de y_pred: 93.25, max de y_pred: 109.74, moyenne de y_pred: 102.03

```

```

R2 score en utilisant r2_score : 0.38
Training RMSE : 5.80

```

R2 isn't very high.

Does this mean the model doesn't explain the data very well?

Let's calculate the correlation matrix between all the numeric variables in the dataframe:

```
1 print(df.corr(numeric_only=True).round(2))
```

	Volume	Weight	CO2
Volume	1.00	0.75	0.59
Weight	0.75	1.00	0.55
CO2	0.59	0.55	1.00



The correlation matrix shows a high correlation between the independent variables (volume and weight). We will fit the model with only one variable at a time and compare the results.

```
1 # Modèle avec uniquement Weight
2 X_weight = df[['Weight']]
3 model_weight = linear_model.LinearRegression()
4 model_weight.fit(X_weight, y)
5 y_pred_weight = model_weight.predict(X_weight)
6
7 print(f"R2 score avec Weight seulement : {model_weight.score(X_weight, y):.2f}")
8 print(f"RMSE avec Weight seulement : {np.sqrt(mean_squared_error(y, y_pred_weight)):.2f}")
```



R2 score avec Weight seulement : 0.30
RMSE avec Weight seulement : 6.13

```
1 # Modèle avec uniquement Volume
2 X_volume = df[['Volume']]
3 model_volume = linear_model.LinearRegression()
4 model_volume.fit(X_volume, y)
5 y_pred_volume = model_volume.predict(X_volume)
6
7 print(f"R2 score avec Volume seulement : {model_volume.score(X_volume, y):.2f}")
8 print(f"RMSE avec Volume seulement : {np.sqrt(mean_squared_error(y, y_pred_volume)):.2f}")
```



R2 score avec Volume seulement : 0.35
RMSE avec Volume seulement : 5.92



Least Squares Loss Function

- **Least Squares Loss** is one of the most commonly used loss functions in **regression** tasks. It measures how close the predicted values are to the actual values by squaring the difference.
- The optimization goal is to find the model parameters that minimize this loss:

Mean Squared Error

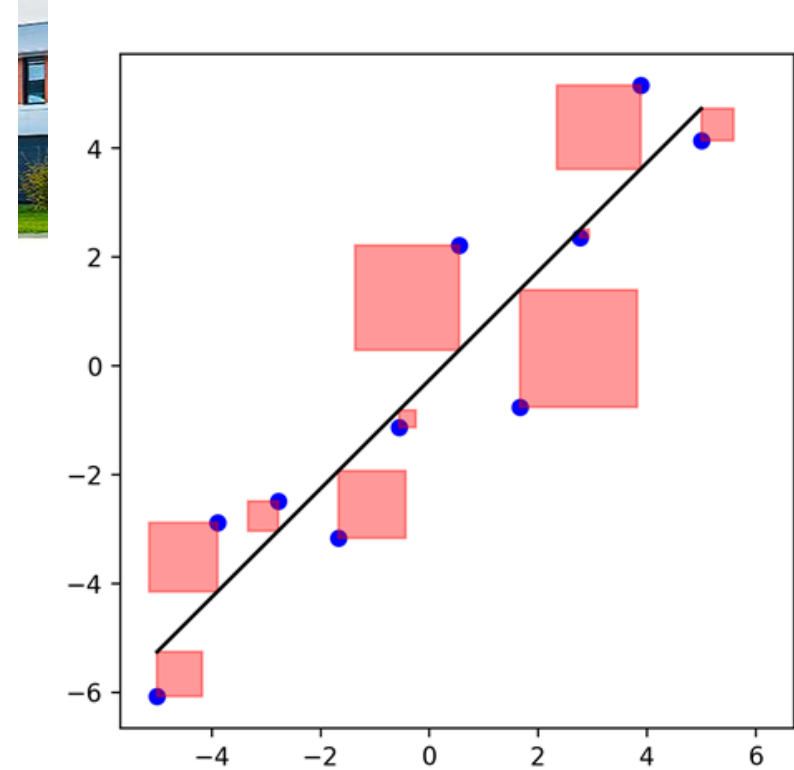
$$C(H) = \frac{1}{2n} \sum_{i=1}^n (H(x_i) - y_i)^2$$

H : parameters of the model

1/2: simplify the derivative of the loss function with respect to the parameters

$$H(x) = w_1 x + w_0 \quad \Rightarrow \quad C(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (w_1 x_i + w_0 - y_i)^2$$

? ?

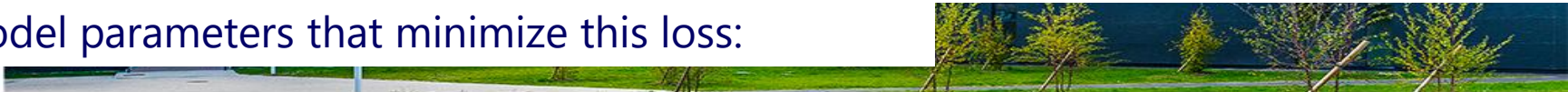


Determining the best parameters (w_0, w_1) for the hypothesis function H amounts to finding the **best straight line** that **minimizes the sum of all unit errors**.

Two methods to solve this problem:

1 - Analytical method (direct computation): The least squares regression method provides an exact solution.

The optimization goal is to find the model parameters that minimize this loss:



$$\begin{cases} \frac{\partial C(w_0, w_1)}{\partial w_0} = 0 \\ \frac{\partial C(w_0, w_1)}{\partial w_1} = 0 \end{cases} \Rightarrow \begin{cases} \sum_{i=1}^n (w_1 x_i + w_0 - y_i) = 0 \\ \sum_{i=1}^n (w_1 x_i + w_0 - y_i) x_i = 0 \end{cases}$$

$$\Rightarrow \begin{cases} w_1 = \frac{\sum_{i=1}^n (y_i - \bar{y}) x_i}{\sum_{i=1}^n (x_i - \bar{x}) x_i} \\ w_0 = \bar{y} - w_1 \bar{x} \end{cases}$$

$$w_0 = \frac{1}{n} \sum_{i=1}^n y_i - w_1 \frac{1}{n} \sum_{i=1}^n x_i$$

2 - Numerical solution method: Gradient descent is an iterative approach to minimize the cost function.

The use of vectors and matrices avoids explicit loops and relies on algorithms optimized for matrix calculations.

$$H(X) = Xw$$

$$X = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \dots & \dots \\ 1 & x_n \end{bmatrix} \quad H(X) = \begin{bmatrix} H(x_1) \\ H(x_2) \\ \dots \\ H(x_n) \end{bmatrix} \quad w = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

$$\Rightarrow H(X) = \begin{bmatrix} H(x_1) \\ H(x_2) \\ \dots \\ H(x_n) \end{bmatrix} = Xw = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \dots & \dots \\ 1 & x_n \end{bmatrix} \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} = \begin{bmatrix} w_0 + w_1 x_1 \\ w_0 + w_1 x_2 \\ \dots \\ w_0 + w_1 x_n \end{bmatrix}$$

$n \times 2 \quad 2 \times 1 \quad n \times 1$



Gradient Descent Update Rules

The **learning rate alpha** is the size of the step Gradient Descent takes all the way until it reaches the global minimum, and it directly impacts the **performance** of the algorithm.

$$\underbrace{w^{(t+1)}}_{\text{position of next iteration}} = \underbrace{w^{(t)}}_{\text{position of previous step}} - \underbrace{\alpha \nabla f(w^{(t)})}_{\text{learning rate} \times \text{step}}$$

Small Learning Rate

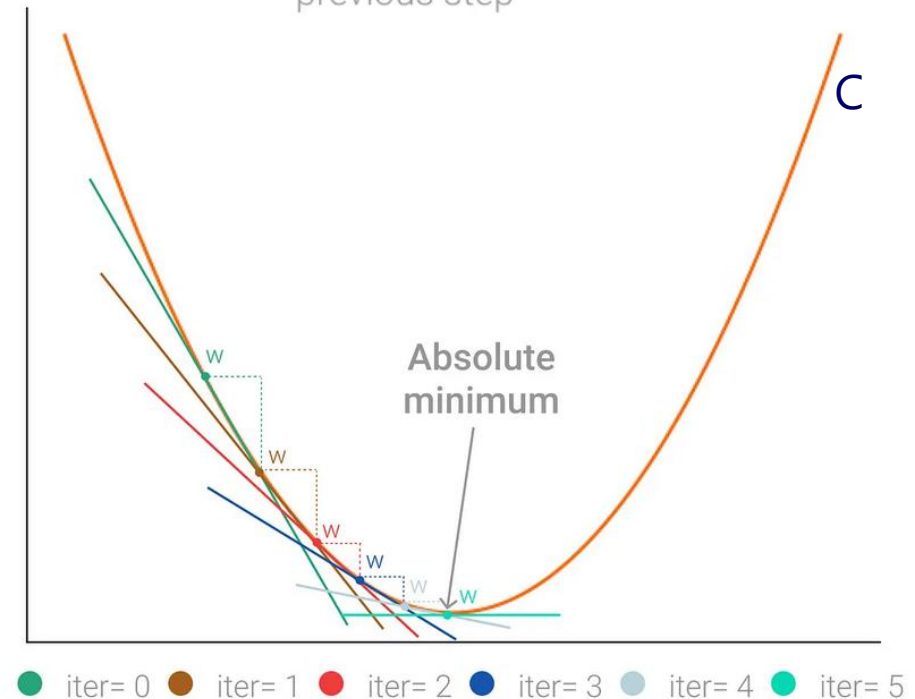
- Small steps
- Gradient descent takes longer to converge

Large Learning Rate

- Big steps
- May step over global minimum several times
- Gradient Descent may never converge

$$C(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (w_1 x_i + w_0 - y_i)^2$$

- If $\partial C / \partial w_j > 0 \rightarrow$ **increasing w_j will increase the cost function $C \rightarrow$ decrease w_j to reduce the cost function.**
- If $\partial C / \partial w_j < 0 \rightarrow$ **increasing w_j will decrease the cost function $C \rightarrow$ increase w_j to reduce the cost function.**





The least squares regression method

$$H(X) = Xw$$

$$C(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (H(x_i) - y_i)^2 \quad \rightarrow \quad C(w) = \frac{1}{2n} \|Xw - y\|^2 = \frac{1}{2n} (Xw - y)^T (Xw - y)$$



$$(Xw - y)^T (Xw - y) = (w^T X^T - y^T)(Xw - y) = w^T X^T Xw - w^T X^T y - y^T Xw + y^T y$$



$$\frac{\partial (w^T X^T Xw - 2w^T X^T y + y^T y)}{\partial w} = 0 \Leftrightarrow 2X^T Xw - 2X^T y = 0$$

$$X^T Xw = X^T y$$



$$w = (X^T X)^{-1} X^T y$$



Gradient descent is a key optimization algorithm in machine learning, allowing model parameters to be adjusted to minimize a cost function.

Step 1: Initialize (w_0, w_1)

$$\begin{cases} w_0 := w_0 - \alpha \frac{\partial C}{\partial w_0} \\ w_1 := w_1 - \alpha \frac{\partial C}{\partial w_1} \end{cases} \quad \begin{cases} C(\mathbf{w}) = \frac{1}{2n} (\mathbf{X}\mathbf{w} - \mathbf{y})^T (\mathbf{X}\mathbf{w} - \mathbf{y}) \\ \frac{\partial C}{\partial \mathbf{w}} = 2\mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y}) \end{cases}$$

$$\begin{cases} w_0 := w_0 - \frac{\alpha}{n} \sum_{i=1}^n (w_1 x_i + w_0 - y_i) \\ w_1 := w_1 - \frac{\alpha}{n} \sum_{i=1}^n (w_1 x_i + w_0 - y_i) x_i \end{cases} \quad \rightarrow \quad \boxed{\mathbf{w} := \mathbf{w} - \frac{\alpha}{n} \mathbf{X}^T (\mathbf{X}\mathbf{w} - \mathbf{y})}$$

$$\rightarrow C(w_0, w_1) = \frac{1}{2n} \sum_{i=1}^n (w_1 x_i + w_0 - y_i)^2 = 0$$

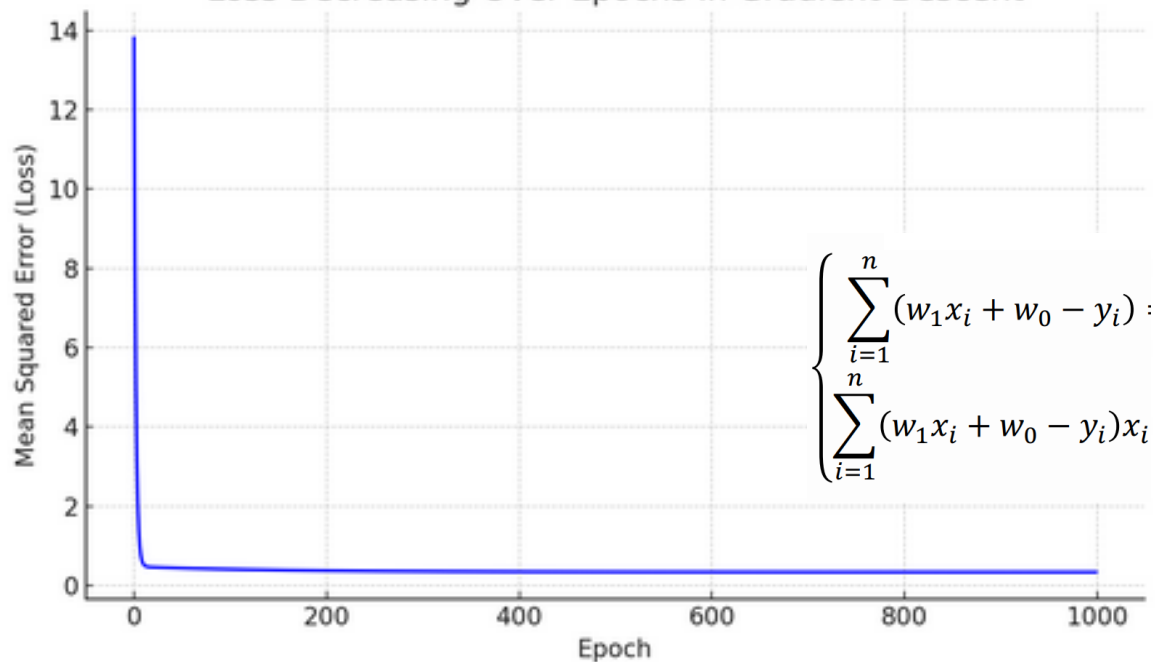
$\alpha > 0$ is the learning rate, a parameter to be chosen (typically small, such as 0.01 or 0.001)

Step 2 is repeated until a stopping criterion is reached (for example, a fixed number of iterations is reached or the variation of the (w_0, w_1) becomes very small).



- Learns the best fit line by minimizing the squared error
- The red line shows the learned linear model

Loss Decreasing Over Epochs in Gradient Descent



loss (Mean Squared Error) decreases over time as gradient descent optimizes the model. The curve typically starts high and flattens out, indicating that the model is converging to the best-fit line.

```
# Sample data
X = np.array([1, 2, 3, 4, 5])
y = np.array([1.5, 3.7, 2.8, 4.5, 5.0])

# Initialize parameters
theta_0 = 0 # intercept
theta_1 = 0 # slope
alpha = 0.01 # learning rate
epochs = 1000
m = len(X)

# Gradient Descent
for _ in range(epochs):
    y_pred = theta_0 + theta_1 * X
    error = y - y_pred
    grad_0 = -2 * np.sum(error) / m
    grad_1 = -2 * np.sum(error * X) / m
    theta_0 -= alpha * grad_0
    theta_1 -= alpha * grad_1

print(f"Learned parameters: theta_0 = {theta_0:.2f}, theta_1 = {theta_1:.2f}")

# Plotting
plt.scatter(X, y, label='Data')
plt.plot(X, theta_0 + theta_1 * X, color='red', label='Fitted Line')
plt.xlabel('X')
plt.ylabel('y')
plt.title('Linear Regression via Gradient Descent')
plt.legend()
plt.grid(True)
plt.show()
```

$$H(x) = w_1 x + w_0$$

$$\begin{cases} w_0 := w_0 - \alpha \frac{\partial C}{\partial w_0} \\ w_1 := w_1 - \alpha \frac{\partial C}{\partial w_1} \end{cases}$$



The End